



MANIPAL
UNIVERSITY JAIPUR
University under Section 2(f) of the UGC Act

INTERNAL ASSIGNMENT

Course Code & Name: DCA2106 – Java Programming

Program: BCA (Bachelor of Computer Applications)

Semester: III

Session: July/September 2025

Name: Hirdyansh Varshney

Roll No.: 2414500172

Course: BCA

Subject: Java Programming

Assignment: Set-I & Set-II

Submission Date: 03-11-2025

Submitted To:

Manipal University Jaipur
Centre for Distance and Online Education

Set – 1

Question 1 :

Answer :

Memory management is a fundamental concept in programming, directly influencing application performance, stability, and resource utilization. Java and C++—while both object-oriented languages—approach memory management with distinct philosophies. Understanding these differences is crucial for developers working across platforms.

Manual vs Automatic Memory Handling

C++ provides manual memory management, giving developers full control over allocation and deallocation. Memory is allocated using `new` and released using `delete`. This flexibility allows for optimized performance but introduces risks such as memory leaks, dangling pointers, and double deletions. Java, in contrast, automates memory management through its built-in garbage collector. Developers create objects using `new`, but do not explicitly free memory. The Java Virtual Machine (JVM) handles cleanup, reducing human error and improving reliability.

Garbage Collection in Java

Garbage collection (GC) is a key feature of Java's memory model. It automatically identifies objects that are no longer reachable and reclaims their memory. Java uses algorithms like Mark-and-Sweep, Generational GC, and G1 GC to optimize this process. GC runs in the background, minimizing disruption to application performance. This automation simplifies development and enhances safety, especially in large-scale applications.

Pointer Safety and References

C++ allows direct pointer manipulation, enabling access to memory addresses. While powerful, this can lead to unsafe operations and undefined behavior. Java eliminates pointers entirely, using references instead. This design choice enhances security and prevents accidental memory corruption. Java's reference model also supports strong, weak, and soft references, offering nuanced control over object lifecycle.

Stack vs Heap Allocation

Both languages use stack and heap memory, but their usage differs. In C++, developers decide whether variables reside on the stack or heap. Objects created with `new` go to the heap, while local variables typically use the stack. Java stores primitive types on the stack and objects on the heap. The JVM dynamically manages this allocation, optimizing memory usage based on runtime conditions.

Resource Cleanup Mechanisms

C++ uses destructors to release resources when an object goes out of scope. This is deterministic and gives precise control. Java previously supported the `finalize()` method, but it's deprecated due to unpredictability. Instead, Java encourages using `try-with-resources` and implementing `Auto Closeable` for resource management, ensuring timely cleanup without relying on GC timing.

Conclusion

Java's memory management prioritizes safety and developer convenience through garbage collection and reference-based access. C++ offers granular control, which is beneficial for system-level programming but demands careful handling. Choosing between the two depends on project needs—Java suits enterprise applications, while C++ excels in performance-critical systems. Understanding these differences empowers developers to write efficient, secure, and maintainable code.

Question 2:

Answer:

In Java, both interfaces and abstract classes are used to achieve abstraction, allowing developers to define methods without implementing them immediately. However, they differ in structure, purpose, and use cases.

Definition and Purpose

An interface is a blueprint for a class, containing method declarations without any implementation. It defines a contract that implementing classes must fulfill. Interfaces are ideal for defining capabilities that can be shared across unrelated classes.

An abstract class, on the other hand, is a partially implemented class. It can contain both abstract methods (without body) and concrete methods (with body). Abstract classes are used when there is a common base behavior that multiple subclasses should inherit.

Syntax and Structure

- Interface: All methods are implicitly public and abstract (until Java 8, which introduced default and static methods).
- Abstract Class: Can have any access modifier, constructors, instance variables, and implemented methods.

Multiple Inheritance

Java does not support multiple inheritance with classes, but it allows a class to implement multiple interfaces. This makes interfaces a powerful tool for designing flexible systems. Abstract classes cannot be extended by more than one class, limiting inheritance.

Fields and Constants

Interfaces can only have static final constants. Abstract classes can have instance variables, which can be inherited and modified by subclasses.

Constructors

Interfaces cannot have constructors because they don't maintain state. Abstract classes can have constructors, allowing them to initialize fields and perform setup tasks.

Use Cases

Use an interface when:

- You want to define a common behavior across unrelated classes.
- You need multiple inheritance.
- You're designing APIs or frameworks.

Use an abstract class when:

- You want to share code among closely related classes.
- You need constructors or non-static fields.

- You want to provide default behavior that subclasses can override.

Example

```
interface Drawable {  
    void draw();  
}  
  
abstract class Shape {  
    int x, y;  
    abstract void draw();  
    void moveTo(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
}
```

Conclusion

Interfaces and abstract classes serve different purposes in Java's object-oriented design. Interfaces promote flexibility and decoupling, while abstract classes offer shared behavior and structure. Choosing between them depends on the design needs—interfaces for capability sharing, abstract classes for inheritance and partial implementation.

Question 3:

Answer:

In Java, the String class provides several methods for manipulating text. Among them, `replace()` and `replaceAll()` are commonly used to substitute characters or substrings. Although they may appear similar, they differ in functionality and internal implementation.

1. `replace()` Method

The `replace()` method is used to replace all occurrences of a character or a sequence of characters with another. It works with character literals or `CharSequence` and performs a literal replacement, meaning it does not interpret regular expressions.

Syntax:

```
String replace(char oldChar, char newChar)
```

```
String replace(CharSequence target, CharSequence replacement)
```

Example:

```
public class ReplaceExample {  
    public static void main(String[] args) {  
        String text = "apple banana apple";  
        String result = text.replace("apple", "orange");  
        System.out.println(result); // Output: orange banana orange  
    }  
}
```

In this example, all literal occurrences of "apple" are replaced with "orange".

2. replaceAll() Method

The `replaceAll()` method replaces each substring that matches a regular expression with a specified replacement. It is more powerful and flexible, especially when pattern matching is required.

Syntax:

```
String replaceAll(String regex, String replacement)
```

Example:

```
public class ReplaceAllExample {  
    public static void main(String[] args) {  
        String text = "apple123banana456apple";  
        String result = text.replaceAll("\\d+", "#");  
        System.out.println(result); // Output: apple#banana#apple  
    }  
}
```

Here, `\\d+` is a regular expression that matches one or more digits. All digit sequences are replaced with "#".

Key Differences

Feature	replace()	replaceAll()
Input Type	CharSequence or char	Regular Expression (regex)
Regex Support	No	Yes
Use Case	Simple replacements	Pattern-based replacements
Performance	Slightly faster for literals	Slightly slower due to regex processing

Conclusion

While both methods are used for replacing content in strings, `replace()` is ideal for straightforward, literal substitutions. In contrast, `replaceAll()` is suited for complex pattern-based replacements using regular expressions. Choosing the right method depends on the nature of the transformation required.

Set – 2

Question 4:

Answer:

Event-Driven Programming

Event-driven programming is a programming paradigm where the flow of the program is determined by events such as user actions (clicks, key presses), sensor outputs, or messages from other programs. In this model, the application remains idle until an event occurs, at which point an event handler is triggered to respond.

This approach is commonly used in graphical user interface (GUI) applications, where users interact with buttons, menus, and forms. For example, in Java Swing, clicking a button can trigger an `ActionListener` that performs a specific task. Event-driven systems are typically built around a main loop that listens for events and dispatches them to appropriate handlers.

The key advantage of this model is its responsiveness and flexibility. It allows developers to build interactive applications that react to user input in real time. It also promotes modularity, as each event can be handled by a separate function or object.

Difference from Procedural Programming

Procedural programming follows a linear, top-down approach where the program executes a predefined sequence of instructions. It is structured around procedures or functions, and control flows from one statement to the next in a predictable order.

In contrast, event-driven programming is non-linear. The program does not follow a fixed path but instead waits for events to dictate the next action. This makes it more suitable for applications that require user interaction or asynchronous behavior.

While procedural programming is ideal for tasks with a clear sequence, event-driven programming excels in environments where the flow depends on external inputs. Understanding both paradigms helps developers choose the right approach based on the nature of the application.

Question 5:

Answer:

Purpose and Functionality of JList

In Java Swing, JList is a user interface component that displays a list of items in a vertical format. It allows users to select one or more items from the list, making it useful for applications that require multiple choices or options.

The component is built on top of the ListModel, which manages the data shown in the list. Developers can use either a default model or create a custom one to handle dynamic data. JList also supports custom rendering, allowing each item to be displayed with specific formatting or icons.

Here's a basic example:

```
String[] colors = {"Red", "Green", "Blue"};
JList<String> colorList = new JList<>(colors);
```

This creates a list of color names. You can add a scroll pane to make it scrollable and attach listeners to detect user selections.

Difference from Other List-Type Components

JList stands apart from components like JComboBox and JTable due to its layout and interaction style. While JComboBox presents a dropdown menu and typically allows single selection, JList displays all items openly and supports multiple selections.

JTable, on the other hand, is designed for displaying structured data in rows and columns. It's more complex and suited for spreadsheet-like interfaces. JList is simpler and better for linear data presentation.

Another key difference is that JList does not include built-in editing features, unlike JTable, which allows cell editing. Also, JList focuses on selection rather than data manipulation.

In conclusion, JList is a lightweight and flexible component ideal for displaying selectable lists, while other components serve more specific roles in GUI development.

Question 6:

Answer: In Java, both ArrayList and LinkedList are part of the java.util package and implement the List interface. They are used to store ordered collections of elements, but they differ in internal structure, performance characteristics, and ideal use cases.

An ArrayList is backed by a dynamic array. It allows fast access to elements using their index because it stores elements in contiguous memory locations. However, inserting or deleting elements in the middle of the list can be costly, as it requires shifting elements to maintain order.

A LinkedList, on the other hand, is based on a doubly linked list. Each element is stored in a node that contains references to both the previous and next nodes. This structure makes insertion and deletion operations efficient, especially in the middle of the list, but accessing elements by index is slower because it requires traversal from the beginning or end.

For example, if you need to frequently access elements by index, such as in a search or sorting operation, ArrayList is preferred:

```
ArrayList<String> names = new ArrayList<>();  
names.add("Alice");  
names.add("Bob");  
System.out.println(names.get(1)); // Fast access
```

If your application involves frequent insertions or deletions, especially in the middle of the list, LinkedList is more suitable:

```
LinkedList<String> tasks = new LinkedList<>();  
tasks.add("Task1");  
tasks.add("Task2");  
tasks.add(1, "UrgentTask"); // Efficient insertion
```

Another difference is memory usage. ArrayList uses less memory per element, while LinkedList consumes more due to node overhead. Also, ArrayList is better for scenarios where resizing is infrequent, whereas LinkedList handles dynamic changes more gracefully.

In conclusion, choose ArrayList for fast random access and low memory overhead, and opt for LinkedList when your application demands frequent insertions or deletions.

Understanding their differences helps in writing optimized and maintainable Java programs.

